

Portfolio-Website: Architektur, Code & Deployment

Zusammenfassung: wie die Anwendung technisch aufgebaut ist, wie die wichtigsten Code-Bausteine funktionieren und wie das Deployment auf Azure ablaeuft.

1. Allgemeiner Ablauf - vom Seitenaufruf bis zur Datenbank

Die Anwendung besteht aus zwei getrennten Azure-Ressourcen (Static Web App fuer Frontend, App Service fuer Backend) plus einer PostgreSQL-Datenbank, verbunden ueber eine REST-API per HTTPS.

```
Browser
| GET https://<domain>
v
Azure Static Web App -- liefert statische Angular-Dateien --> Browser
|
| Angular-App macht im Browser XHR-Requests:
| GET https://emre-portfolio-backend-....azurewebsites.net/api/projects
v
Azure App Service (Spring-Boot-Jar, Java 17, eingebetteter Tomcat)
|
| Spring Data JPA / Hibernate via JDBC
v
PostgreSQL-Datenbank
```

Ablauf beim Oeffnen der Seite:

- Der Browser laedt die Angular-App von der Azure Static Web App (reines Static Hosting / CDN, kein eigener Server).
- **staticwebapp.config.json** sorgt dafuer, dass alle Routen (z.B. /admin/dashboard) auf index.html zurueckfallen (Single Page App-Routing) und setzt Security-Header (CSP, X-Frame-Options etc.).
- Die laufende Angular-App ruft ueber HttpClient die Backend-API auf (feste Azure-App-Service-URL aus environment.prod.ts).
- Das Backend (Spring Boot auf Azure App Service) durchlaeuft Security-Filter (CORS, Rate-Limit, JWT), dann Controller -> Service -> Repository (JPA).
- PostgreSQL liefert Daten zurueck, Hibernate mappt sie auf Entities, der Service baut daraus ein DTO, der Controller gibt JSON zurueck.
- Angular verarbeitet das JSON und rendert Projekte/Skills.

Zwei unabhaengige CI/CD-Pipelines (GitHub Actions):

- **azure-static-web-apps-....yml**: baut bei jedem Push auf main das Angular-Frontend (npm ci && ng build --configuration production) und laedt den Ordner dist/myportfolio/browser direkt in die Static Web App hoch (per API-Token).
- **backend-deploy.yml**: laeuft nur bei Aenderungen unter backend/**. Baut mit Maven (mvnw clean package), loggt sich per OIDC (passwortlos) bei Azure ein und deployt das JAR per 'az webapp deploy' auf den App Service.

2. Backend im Detail (Spring Boot)

2.1 Security & JWT-Authentifizierung

- **Stateless JWT-Auth:** kein Session-Cookie (SessionCreationPolicy.STATELESS). Login (POST /api/auth/login) prueft Username/Passwort ueber den AuthenticationManager und gibt bei Erfolg ein signiertes JWT zurueck.
- **JwtService:** signiert Tokens mit HMAC-SHA256. Das Secret kommt zwingend aus der Umgebungsvariable JWT_SECRET - ohne gesetzte Variable startet die App gar nicht (kein unsicherer Fallback).
- **JwtAuthenticationFilter:** liest bei jedem Request den Authorization: Bearer ...-Header, validiert das Token und setzt bei Gueltigkeit den SecurityContext.
- **Autorisierung (SecurityConfig):** GET auf /api/projects/** und /api/skills/** ist oeffentlich, /api/admin/** braucht Rolle ADMIN, alles andere ist standardmaessig authentifiziert.
- **LoginRateLimitFilter:** einfacher In-Memory-Brute-Force-Schutz (max. 8 Login-Versuche pro IP in 5 Minuten). Da Azure App Service als Reverse-Proxy davorsteht, wird die echte Client-IP aus dem Header X-Forwarded-For gelesen statt aus request.getRemoteAddr().
- **AdminInitializer:** legt beim ersten Start automatisch einen Admin-User an, falls noch keiner existiert (Zugangsdaten aus Umgebungsvariablen, Passwort gehasht per Delegating-PasswordEncoder / bcrypt).
- **CorsConfig:** erlaubte Frontend-Origins kommen aus der Umgebungsvariable CORS_ALLOWED_ORIGINS (lokal Standard: localhost:4200). Muss auf die tatsaechliche Static-Web-App-Domain zeigen, sonst blockt der Browser die Requests.

2.2 Datenbank & Migrationen (Flyway)

- **Flyway** verwaltet das DB-Schema versioniert ueber SQL-Dateien (V1 bis V6 in db/migration).
- **ddl-auto=validate:** Hibernate erstellt nie selbst Tabellen, sondern prueft nur, ob die Java-Entities zum (von Flyway erzeugten) Schema passen - verhindert Drift zwischen Code und Datenbank.
- Tabellen: admins, projects (+ project_technologies, project_screenshots als @ElementCollection), project_files (Binaerdateien als bytea direkt in der DB, kein Blob-Storage), skills (+ skill_descriptions, skill_technologies).
- Verbindung laeuft ueber die Umgebungsvariablen DB_URL, DB_USERNAME, DB_PASSWORD - als Azure App Service Application Settings hinterlegt, nicht im Code.

2.3 Projekt-Feature (Kernlogik)

- Die Project-Entity generiert automatisch einen eindeutigen Slug aus dem Titel (@PrePersist/@PreUpdate).
- ProjectService.createUniqueSlug() haengt bei Kollisionen -2, -3 usw. an, damit der Slug garantiert eindeutig bleibt.
- Oeffentliche Endpunkte (ProjectController) sind reine Lesezugriffe. Schreibzugriffe laufen ausschliesslich ueber AdminProjectController unter /api/admin/projects, geschuetzt durch die JWT-Rolle ADMIN.

3. Frontend im Detail (Angular, Standalone Components)

- **Lazy-Loading pro Route** (app.routes.ts): jede Seite (Home, AdminDashboard, AdminLogin, ...) wird erst beim Aufruf geladen -> kleineres initiales Bundle.
- **adminGuard:** schuetzt /admin/dashboard clientseitig, indem geprueft wird, ob ein gueltiges (nicht abgelaufenes) Token in sessionStorage liegt.
- **authInterceptor:** haengt den Authorization-Header nur bei Requests an /api/admin/** an; bei einer 401-Antwort wird automatisch ausgeloggt.
- **sessionStorage statt localStorage:** bewusste Entscheidung - das Admin-Token verschwindet beim Schliessen des Tabs.

- **AuthService** dekodiert clientseitig den JWT-Payload (Base64), um das exp-Feld zu prüfen, bevor überhaupt ein Request rausgeht. Spart unnötige 401-Roundtrips; die eigentliche Prüfung bleibt aber serverseitig.

4. Deployment im Detail

4.1 Azure Static Web App (Frontend)

Ein Static Web App (SWA) ist kein eigener Server, sondern ein globaler CDN-Dienst speziell fuer statische Frontends (HTML/CSS/JS, wie sie 'ng build' erzeugt).

- Beim Deploy wird einfach ein Ordner mit fertigen Dateien hochgeladen (frontend/dist/myportfolio/browser). Azure verteilt diese Dateien automatisch ueber mehrere globale Edge-Standorte.
- Es gibt keinen laufenden Prozess, der bezahlt oder gewartet werden muss - kein 'Server', der abstuerzen koennte. Requests werden vom naechstgelegenen CDN-Knoten beantwortet.
- staticwebapp.config.json steuert Verhalten, das sonst ein Webserver uebernehmen wuerde: URL-Rewrites (navigationFallback auf index.html fuer Angular-Routing) sowie globale HTTP-Security-Header.
- SWA koennte auch eine eigene 'Managed API' (Azure Functions) hosten - wird hier bewusst NICHT genutzt (api_location: ""), weil das Backend separat auf App Service laeuft (volle Kontrolle statt Functions-Limits).
- Deployment per API-Token (GitHub-Secret), das bei Erstellung der SWA-Ressource automatisch generiert und ins Repo eingetragen wurde. Kein Build auf Azure-Seite (skip_app_build: true), da selbst mit aktuellem Node gebaut wird.

4.2 Azure App Service (Backend)

App Service ist ein PaaS (Platform-as-a-Service) fuer laufende Serveranwendungen - hier laeuft tatsaechlich ein Prozess dauerhaft, der Requests verarbeitet.

- Ausgewaehlt werden ein App Service Plan (zugrundeliegende Ressourcengroesse: CPU/RAM/Skalierung) und ein Runtime-Stack (hier: Java 17).
- Azure stellt eine Sandbox bereit, die das JAR ausfuehrt (java -jar app.jar), schaltet einen Reverse-Proxy davor (HTTPS-Terminierung, Load Balancing, Health-Checks) und startet den Prozess bei einem Absturz automatisch neu.
- App Settings = Umgebungsvariablen, die Azure beim Start injiziert (DB_URL, DB_PASSWORD, JWT_SECRET, CORS_ALLOWED_ORIGINS), verschluesselt gespeichert, nie im Code.
- Weil der Proxy davor sitzt, sieht Spring Boot nicht die echte Client-IP, sondern die des Proxys - daher liest der LoginRateLimitFilter bewusst X-Forwarded-For.
- Deployment: az webapp deploy --type jar - das JAR wird per Kudu (Azures Deployment-Engine) hochgeladen, der alte Prozess gestoppt, das neue JAR platziert und neu gestartet.

4.3 Spring-Boot-JAR statt Docker-Image

Ein Spring-Boot 'Fat JAR' ist eine einzelne .jar-Datei, die nicht nur den kompilierten Code enthaelt, sondern auch alle Abhaengigkeiten UND einen eingebetteten Tomcat-Server (automatisch erzeugt durch das spring-boot-maven-plugin beim package-Schritt). 'java -jar portfolio-backend.jar' reicht daher aus, um die App inklusive HTTP-Server zu starten - kein separat installierter Application-Server noetig.

Ein Docker-Image waere die Alternative: eine komplette, isolierte Container-Umgebung inklusive eigenem Betriebssystem-Layer.

Warum das JAR hier die pragmatischere Wahl ist:

- Weniger bewegliche Teile: kein Dockerfile pflegen, kein Base-Image aktuell halten, keine eigene Container-Registry.
- Azure App Service kuummert sich bereits um die Laufzeitumgebung: der 'Java 17 on Linux'-Stack ist ein gepflegtes, aktuelles Basis-Image - Patching uebernimmt Microsoft. Ein eigenes Docker-Image wuerde das nur duplizieren.

- Einfachere, schnellere Pipeline: 'mvn package' -> 'az webapp deploy' ohne Docker-Build/-Push spart Zeit und Komplexitaet in CI.
- Fuer eine Single-Instance-App ohne exotische System-Abhaengigkeiten bringt Docker kaum Vorteile - dessen groesster Nutzen (identisches Verhalten inkl. OS-Ebene ueberall) zaehlt sich erst bei Multi-Service-Orchestrierung (Kubernetes, Container Apps) oder Native-Dependencies aus.

4.4 OIDC / Federated Credentials

Der klassische Weg fuer GitHub-Actions-Azure-Login waere ein Service Principal mit Client-Secret - ein Passwort im GitHub-Secret, das potenziell monatelang gueltig und bei einem Leak sofort ausnutzbar ist.

OIDC (OpenID Connect) + Federated Credentials ersetzt dieses langlebige Passwort durch ein Vertrauensverhaeltnis zwischen GitHub und Azure AD, ganz ohne gespeichertes Secret:

- In Azure AD (Entra ID) wird fuer die App-Registrierung eine Federated Credential angelegt: 'Vertraue Tokens, die GitHub fuer genau dieses Repo/diesen Branch ausstellt' (repo:owner/name:ref:refs/heads/main).
- Beim Workflow-Lauf (azure/login@v2 mit id-token: write) fordert GitHub selbst ein kurzlebiges, signiertes JWT von seinem eigenen OIDC-Provider an - es bestaetigt kryptografisch 'das ist wirklich ein Workflow-Run aus Repo X, Branch main'.
- Azure AD prueft dieses Token gegen die hinterlegte Federated Credential. Passt es, stellt Azure AD ein kurzlebiges Access-Token aus (ca. 1 Stunde gueltig), mit dem az webapp deploy arbeitet.
- Am Ende des Jobs (az logout) ist die Session beendet - es existiert nirgends ein dauerhaftes Passwort.

Vorteile gegenueber Secret-basiertem Login:

- Kein Passwort, das geleakt, rotiert oder von Secret-Scannern gefunden werden kann.
- Zugriff strikt gebunden an 'genau dieser Workflow aus genau diesem Repo/Branch'.
- Tokens sind extrem kurzlebig (Minuten bis maximal ca. 1h) statt oft jahrelang unrotierter Secrets.

client-id, tenant-id und subscription-id im Workflow sind dabei nur oeffentliche Identifikatoren (wer ich bin), keine Passwoerter (kein 'hier ist mein Geheimnis').

5. On-Premises VM vs. PaaS

Der Unterschied liegt darin, wer welche Schicht des Software-Stacks verwaltet: Anwendungscode -> Runtime (JVM) -> Betriebssystem -> Virtualisierung -> physische Hardware.

On-Premises VM (bzw. IaaS wie eine Azure VM)

Verantwortlich fuer praktisch alles oberhalb der Virtualisierung:

- OS installieren, patchen, absichern (Firewall, SSH-Haertung, Kernel-CVEs im Blick behalten).
- Java-Runtime installieren und aktuell halten.
- Prozess selbst am Laufen halten (systemd-Service, Neustart bei Absturz, Logging/Log-Rotation selbst einrichten).
- Skalierung selbst bauen (weitere VMs, Load Balancer, Orchestrierung).
- Backups, Monitoring, SSL-Zertifikate - alles Eigenleistung.

Vorteil: maximale Kontrolle. **Nachteil:** maximaler Betriebsaufwand, der nichts mit dem eigentlichen Produkt zu tun hat.

PaaS (Azure App Service)

Es wird nur die oberste Schicht geliefert - der Anwendungscode (das JAR). Alles darunter uebernimmt Azure:

- Azure patcht Betriebssystem und Java-Runtime automatisch.
- Azure sorgt fuer Neustart bei Absturz.
- Skalierung ist ein Klick/Setting statt eigener Infrastrukturarbeit.
- HTTPS/Zertifikate, Reverse-Proxy, Health-Checks sind eingebaut.
- Deployment per 'az webapp deploy' - kein SSH, kein manuelles Stoppen/Austauschen/Starten.

Vorteil: Fokus nur auf Code statt Infrastruktur. **Nachteil:** weniger Kontrolle (keine beliebigen OS-Pakete, kein Kernel-Tuning), Abhaengigkeit von unterstuetzten Runtimes/Mechanismen. Fuer eine Portfolio-Website mit Standard-Stack (Spring Boot + PostgreSQL) ohne exotische Infrastruktur-Anforderungen ist PaaS die deutlich pragmatischere Wahl.

6. Wie wissen GitHub und Azure voneinander?

Es gibt keine automatische Magie, sondern zwei explizite Verknuepfungen, die einmalig eingerichtet wurden.

Schritt 1: git push -> GitHub

Ein ganz normaler Git-Vorgang ohne jeglichen Azure-Bezug. IntelliJ ist dabei nur eine grafische Oberflaeche fuer denselben Vorgang wie im Terminal.

Schritt 2: GitHub erkennt den Push selbst

GitHub Actions ist ein eingebautes Feature von GitHub selbst (kein externer Webhook noetig). GitHub beobachtet jeden Push und vergleicht ihn mit den on:-Triggern in den Workflow-Dateien (.github/workflows/*.yml). Passt Branch und Pfad, startet GitHub selbststaendig einen Workflow-Lauf auf einer eigenen (von GitHub bereitgestellten) VM - komplett getrennt von der Azure-Umgebung.

Schritt 3: Erst im Workflow-Lauf spricht GitHub mit Azure

Verbindung	Wie hergestellt
Frontend (Static Web App)	Deployment-Token als GitHub-Secret, wurde bei Erstellung der SWA-Ressource automatisch generiert und ueber die GitHub-App-Autorisierung ins Repo eingetragen. Wer das Token besitzt, darf in genau diese eine Ressource deployen.
Backend (App Service)	Zweistufig: (1) OIDC/Federated-Credential-Vertrauen in Azure AD ('vertraue Runs aus genau diesem Repo/Branch') plus (2) explizite Angabe von --name und --resource-group im Workflow-YAML, die sagt, welche konkrete Ressource gemeint ist.

Zusammengefasst: jede Verbindung wurde einmal explizit konfiguriert (beim Erstellen der Azure-Ressourcen bzw. beim Einrichten der Secrets) und laeuft danach bei jedem Push stur nach demselben Muster ab.

7. Monitoring in der Azure Static Web App

Der 'Monitoring'-Tab der Static Web App zeigt nur aggregierte Zahlen, keine Einzel-Besucher-Infos:

- **Requests (SiteHits):** Gesamtzahl der HTTP-Anfragen im Zeitfenster - reine Zaehlung ohne Personenbezug.
- **Data out:** Gesamt ausgelieferter Traffic in MB.
- **Function hits:** Aufrufe der SWA-Managed-API-Functions (hier ungenutzt, da das Backend separat auf App Service laeuft).

Diese eingebaute Azure-Monitor-Ansicht zeigt **wie viel** Traffic, aber nicht **wer** zugreift: keine IP-Adressen, keine Standorte, keine Geraete-Infos, kein Verlauf pro Besucher.

Fuer detailliertere Auswertung waeren zusaetzlich noetig:

- **Application Insights (Azure):** einzelne Requests inkl. IP, User-Agent, Referrer, grober Standort, Ladezeiten. Eigener, separat zu aktivierender Dienst.
- **Web-Analytics-Tool** (z.B. Plausible, Google Analytics, Matomo): client-seitig eingebunden.
- **App Service Log Stream / Access-Logs:** serverseitige Zugriffslogs mit Client-IP.

Datenschutz-Hinweis: Da bereits eine Impressum- und Datenschutz-Seite existiert (EU/DE-Kontext), greift bei Erfassung personenbezogener Daten (IP-Adressen, Tracking-Cookies) die DSGVO - je nach Tool und Konfiguration waere ein Cookie-/Consent-Banner sowie eine Erwaehnung in der Datenschutzerklaerung noetig. Reine aggregierte Server-Metriken wie im Monitoring-Tab sind davon nicht betroffen.

8. Azure API Management - wofuer und warum hier nicht noetig

Azure API Management (APIM) ist ein Gateway/Proxy, der vor Backend-APIs gestellt wird: Clients sprechen dann nicht mehr direkt mit dem Backend, sondern mit dem APIM-Gateway, das Requests weiterleitet.

Typische Aufgaben von APIM:

- Zusammenfassen mehrerer APIs unter einer einheitlichen, versionierten Oberflaeche.
- Rate Limiting / Throttling zentral konfigurierbar.
- API-Keys / Subscriptions fuer kontrollierten Zugriff externer Entwickler.
- Request/Response-Transformation ohne Code-Aenderung.
- Zentrales Monitoring/Analytics ueber alle APIs hinweg.
- Entwickler-Portal mit automatisch generierter API-Dokumentation.

Fuer dieses Projekt nicht noetig: es gibt nur eine einzelne, simple API mit wenigen Endpunkten, die ausschliesslich vom eigenen Angular-Frontend konsumiert wird. Kein Bedarf an mehreren zusammengefuehrten Services, externen Drittanbietern mit eigenen API-Keys oder komplexem Versioning. APIM lohnt sich erst bei groesseren Systemen mit mehreren Microservices oder wenn die API Dritten als Produkt angeboten wird - und kostet zudem laufend Geld.

9. Die API dieses Projekts im Detail

Eine klassische REST-API (Spring Boot), erreichbar unter <https://emre-portfolio-backend-....azurewebsites.net/api>. Vier Bereiche: Projects, Skills, Auth, jeweils mit öffentlichem Lese-Teil und geschütztem Admin-Teil.

Oeffentlich (kein Login, nur GET)

Methode	Pfad	Zweck
GET	/api/projects	alle Projekte (Home-Page)
GET	/api/projects/{slug}	ein Projekt anhand des URL-Slugs
GET	/api/skills	alle Skills
GET	/api/skills/{id}	ein Skill nach ID

Auth

Methode	Pfad	Zweck
POST	/api/auth/login	Admin-Login, gibt bei Erfolg ein JWT zurueck

Admin (geschuetzt: Authorization: Bearer <JWT> + Rolle ADMIN)

Methode	Pfad	Zweck
POST	/api/admin/projects	neues Projekt anlegen
PUT	/api/admin/projects/{id}	Projekt bearbeiten
DELETE	/api/admin/projects/{id}	Projekt loeschen
POST	/api/admin/skills	neuen Skill anlegen
PUT	/api/admin/skills/{id}	Skill bearbeiten
DELETE	/api/admin/skills/{id}	Skill loeschen

Projekt-Objekt (ProjectRequest / ProjectResponse)

```
{
  "title": "string",
  "shortDescription": "string (max 500)",
  "fullDescription": "string (Freitext)",
  "imageUrl": "string",
  "screenshots": ["url1", "url2"],
  "githubUrl": "string",
  "liveUrl": "string",
  "downloadUrl": "string",
  "videoUrl": "string",
  "featured": true,
  "technologies": ["Angular", "Spring Boot"]
}
```

Server generiert automatisch: id, slug (aus dem Titel), createdAt, updatedAt.

Skill-Objekt (SkillRequestDto)

```
{
  "name": "string",
  "iconUrl": "string",
  "description": ["Punkt 1", "Punkt 2"],
  "projectTechnologies": ["Java", "Angular"]
}
```

Zusammenspiel mit dem Frontend

- Öffentliche Endpunkte ruft ProjectService/SkillService in Angular ohne Auth-Header auf.
- Admin-Endpunkte werden vom authInterceptor automatisch mit dem JWT aus sessionStorage versehen (nur für URLs unter /api/admin/).
- SecurityConfig im Backend erzwingt serverseitig dieselbe Trennung - clientseitige und serverseitige Regeln spiegeln sich bewusst.

Die API ist bewusst schlank gehalten: kein API-Gateway, keine externen Consumer, nur das eigene Angular-Frontend als Client.